

理器已经移到了用户态，因此 `spawn` 并不是系统调用，而是处于用户态的系统服务提供的接口。在 `spawn` 执行过程中，进程管理器会使用必要的系统调用与内核交互，最终完成进程创建的功能。

代码片段 6.14 ChCore 中进程创建 (`spawn`) 的伪代码实现

```

1 int spawn(char *path, ...)
2 {
3     // 根据指定的文件路径，加载需要执行的文件（同时创建相关物理内存对象 PMO）
4     struct user_elf *elf = readelf(path);
5
6     // 进程管理器获取一个新的 pid 作为进程标识符
7     int pid = alloc_pid();
8     // 创建进程（包括创建 PCB、虚拟地址空间、处理器上下文和内核栈）
9     int new_process_cap = create_cap_group(pid, path, ...);
10
11     // 为用户栈创建物理内存对象 PMO
12     int stack_cap = create_pmo(...);
13     // 利用数组构造初始执行环境
14     char init_env[ENV_SIZE];
15     construct_init_env(init_env, elf, ...);
16     // 更新执行环境到栈对应的 PMO 中
17     write_pmo(stack_cap, init_env, ...);
18
19     // 构建请求，用于映射栈对应的 PMO
20     struct pmo_map_request requests[MAX_REQ_SIZE];
21     add_request(requests, stack_cap, STACK_VADDR, ...);
22     // 构建请求，用于映射可执行文件中需要加载的段对应的 PMO
23     for (struct user_elf_seg seg: elf->loadable_segs)
24         add_request(requests, seg.pmo, seg.p_vaddr, ...);
25
26     // 完成上述 PMO 的实际映射
27     map_pmos(new_process_cap, requests, ...);
28     // 完成其他部分的初始化（包括 proc 结构体）并返回
29     ...
30 }

```

那么，`spawn` 的执行过程中到底有哪些部分是在内核中完成的呢？图 6.5 展现了 `spawn` 函数的控制流在用户态和内核态之间切换的全过程。注意，虽然负责解析 ELF 文件的 `readelf` 函数主要在用户态执行，但它需要 `create_pmo`、`map_pmo` 等系统调用来创建新的物理内存对象，用于保存之后需要映射的程序节。从图中不难看出，当需要分配更多内核资源，或是需要对内核资源进行修改时，就要使用系统调用进入内核，其他功能都可以在用户态完成。如果要实现其他功能，也可以使用相似的方法。例如对于进程退出接口 `exit`，只需要使用相应的系统调用释放创建的对象，而与用户态 PCB 相关的部分（如进程标识符）就直接在用户态释放即可。

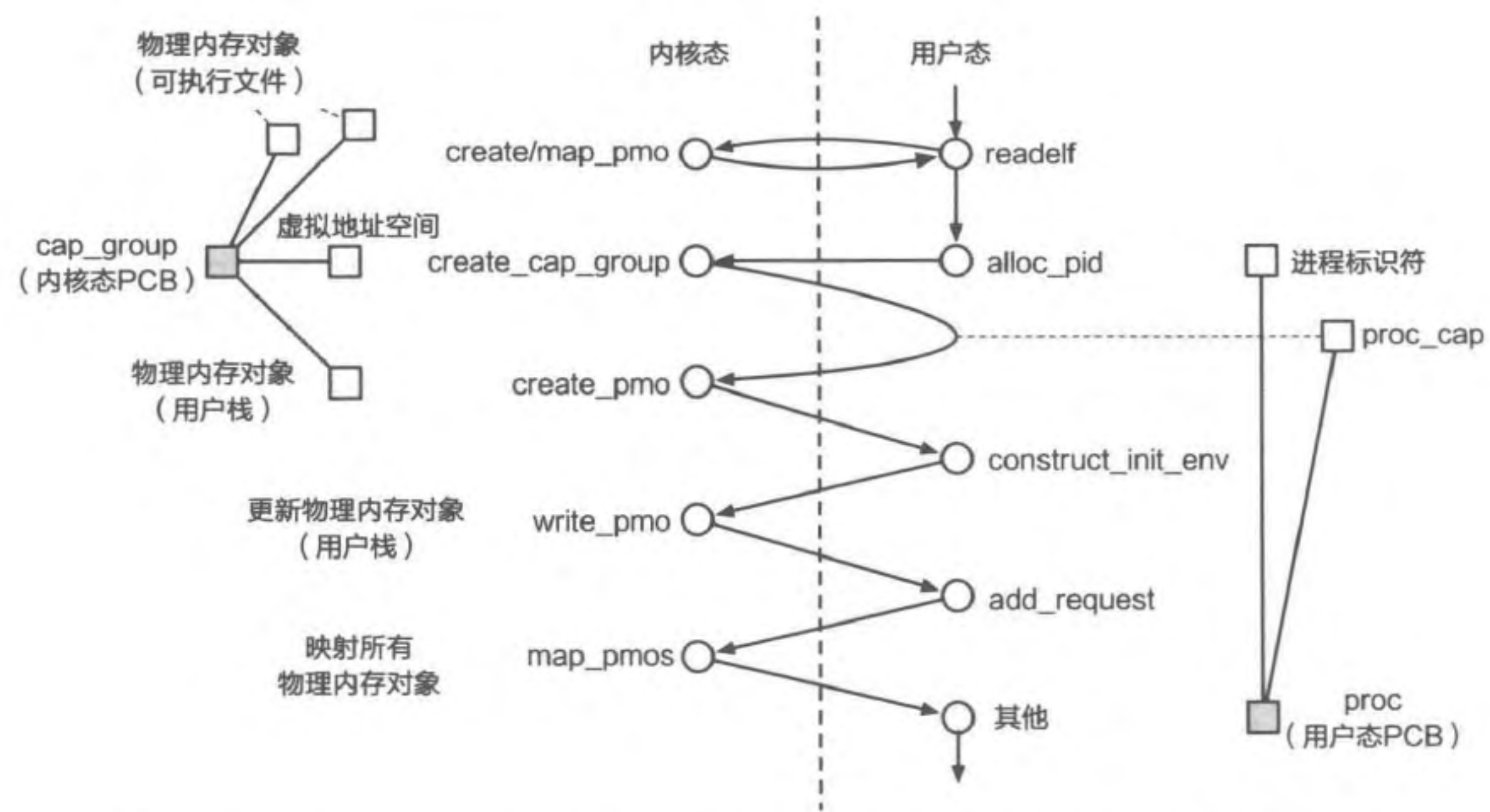


图 6.5 ChCore 的 spawn 函数控制流（圆圈表示执行的函数，方块表示新创建的数据结构）

6.3 案例分析：Linux 的进程创建

本节主要知识点

- ❑ Linux 中最为经典的进程创建接口 fork 是如何实现的？
- ❑ fork 有哪些优点和缺点？
- ❑ Linux 中还有哪些创建进程的方法？它们都有什么特点？

本章已经从必要的功能出发，介绍了进程创建、退出、等待、睡眠相关系统调用的实现方法。而在现代操作系统中，为了满足应用的多样化需求，系统调用往往也是多样化的。对于同一个功能，操作系统可能会提供多种系统调用，它们具有不同的特点，并在不同的历史阶段和应用场景中扮演着重要角色。本节将以进程创建这一功能为例，介绍常见的系统调用及其发展历史。

6.3.1 经典的进程创建方法：fork

fork 是最为经典的进程创建方法，其使用历史非常悠久。早在 20 世纪 60 年代，由加州大学伯克利分校主导的分时系统研究项目 Genie 首先使用 fork 作为系统调用^[1]创建进程，而 UNIX 也沿用了这个设计^[2]。直至今日，fork 仍然广泛地应用于 shell、Web 服务器、浏览器等场景中。

代码片段 6.15 用一个简单的例子展示了 `fork` 的用法（略去了错误处理的分支）。与前面介绍的 `process_create` 相比，`fork` 不会从头创建进程，而是以当前进程（父进程）为基础，创建一份从用户视角看一模一样的拷贝（子进程）。例如在代码片段 6.15 的例子中，由于父进程已经将 `x` 赋值为 42，因此子进程中 `x` 也为 42。

代码片段 6.15 使用 `fork` 的简单例子

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     int x = 42;
8     int ret = fork();
9     if (ret == 0) {
10         // 子进程
11         printf("child: x=%d\n", x);
12     } else {
13         // 父进程
14         printf("parent: x=%d\n", x);
15     }
16     return 0;
17 }
```

在调用完 `fork` 之后，用户可能想让父进程和子进程执行不同的任务。但父进程和子进程一模一样，该如何分辨呢？实际上，内核在 `fork` 的返回值上为父、子进程制造了一点不同：如果 `fork` 顺利完成，那么它在子进程中的返回值为 0，而在父进程中的返回值为子进程的标识符。由于在 `fork` 完成后，父进程和子进程的 PC 寄存器也完全相同，都指向调用 `fork` 的下一条指令，因此它们都会从 `fork` 调用中返回（即“调用一次，返回两次”），并可利用返回值的不同设计不同的工作逻辑（如例子中打印不同的内容）。

如果执行代码片段 6.15，我们会发现其输出可能是不确定的。有时会得到：

```
child: x=42
parent: x=42
```

有时则会得到：

```
parent: x=42
child: x=42
```

这是因为虽然父、子进程完全一样，但它们在 `fork` 之后的执行彼此独立，先后顺序完全由内核决定，因而会产生不确定的输出结果。

从上述例子可以看出，`fork` 的接口相当简单，不接收任何参数，只返回一个整型变量。`fork` 的实现与 `process_create` 相比也比较简单，如代码片段 6.16 所示，除了创建 PCB 和初始化内核栈以外，其他部分的初始化都是从现有进程（父进程）简单获取

一份拷贝（本节均假设 PCB 为第一版结构，因此省略了进程标识符的赋值部分）。据 C 语言之父 Dennis Ritchie 介绍，fork 在早期计算机 PDP-7 上仅需要 27 行汇编代码即可实现^[3]，这种简单创建进程的方法使 fork 受到了欢迎。

代码片段 6.16 fork 的伪代码实现

```
1 int fork(void)
2 {
3     // 创建一个新的 PCB，用于管理新进程
4     struct process *new_proc = alloc_process();
5     // 虚拟内存初始化：初始化页表基地址
6     new_proc->vmSPACE->pgtbl = alloc_page();
7     // 虚拟内存初始化：将当前进程（父进程）PCB 中的页表完整拷贝一份
8     copy_vmSPACE(new_proc->vmSPACE, curr_proc->vmSPACE);
9
10    // 内核栈初始化
11    init_kern_stack(new_proc->stack);
12    // 上下文初始化：将父进程 PCB 中的处理器上下文完整拷贝一份
13    copy_context(new_proc->ctx, curr_proc->ctx);
14    // 返回
15 }
```

由于 fork 只能创建现有进程的拷贝，为了能创建对应新应用程序的进程，操作系统还提供了 exec 系统调用，在 fork 的基础上载入新的可执行文件并执行。exec 的实现与 process_create 的功能相似，但它不需要创建全新的 PCB，只需要在 fork 创建的进程基础上载入新的可执行文件，重新初始化 PCB 中的内容（如虚拟地址空间 vmSPACE），并根据应用提供的参数准备运行环境。fork 和 exec 的组合将进程的创建分为职责清晰的两部分；fork 从现有进程出发，通过拷贝创建一个新的可运行进程，为进程搭建了“骨架”；而 exec 则填入了实际需要运行的可执行文件、虚拟内存映射、应用参数等，为进程添加了“血肉”。这种解耦的设计也给进程管理留下了较大空间：在使用 fork 创建子进程后，程序可以在调用 exec 前对子进程进行各种设定，例如限制子进程可以使用的资源。

fork 的诞生距今已有近 60 年的时间，可以说是“年近花甲”。在过去的 60 年里，计算机发生了翻天覆地的变化，这些变化也使得 fork 的设计受到了诸多挑战。

第一，fork 已经变得过于复杂。尽管 fork 的接口依然保持着简洁的风格，但随着操作系统支持的功能越来越多，fork 的实现也越发复杂。由于 fork 的默认语义是构造与父进程一样的拷贝，因此它会使得子进程与父进程共享各种类型的状态，fork 实现的正确性因而与这些状态紧密相关。每当操作系统为进程添加新功能、扩展 PCB 的结构时，就必须考虑是否需要对 fork 的实现做相应修改，fork 在实现过程中需要考虑的特殊情况也就越来越多。实际上，POSIX 标准已经列出了调用 fork 时的 25 种特殊情况，而很多特殊情况都需要开发者小心规避，这足以体现 fork 的复杂性和使用的不便。

性（一个典型的例子是多线程，详见 6.5.4 节）。另一方面，fork 的实现与进程、内存管理等模块耦合程度过高，也不利于内核的代码维护。

第二，fork 的性能太差。由于 fork 需要创建一份原进程的拷贝，因此原进程包含的状态越多，fork 的性能就越差。在过去，fork 也许只要应付内存规模较小的 shell 程序；而到了今天，大内存应用已经十分普遍。尽管第 4 章介绍的写时拷贝技术可以大幅减少虚拟内存初始化过程中的内存拷贝，但对于内存需求较大的应用来说，就连建立页表中的内存映射都需要耗费大量时间，fork 的效率已经满足不了它们的需要了。

第三，fork 存在潜在安全漏洞。fork 建立的父进程与子进程之间的联系可能会成为攻击者的重要切入点。通常来说，同一个程序每次启动的虚拟地址空间布局是随机的，这为攻击者带来了一定的困难：为了实施攻击，攻击者每次都需要先弄清楚进程的虚拟地址空间布局，再实施相应的攻击。但是，fork 创建的子进程与父进程的虚拟地址空间布局完全相同，这就为攻击者扫除了随机性带来的障碍。攻击者一旦弄清楚了父进程的虚拟地址空间布局，就知晓了所有由 fork 创建的子进程的地址空间布局，因此直接实施攻击即可。由于安全特性在计算机中的地位今非昔比，安全问题也对 fork 的地位造成了威胁。

除了以上三点，研究人员还指出了更多 fork 的缺点，比如可扩展性差、与异构硬件不兼容、线程不安全等^[4]。因此，Linux 也提出了针对 fork 的多种替代方案，包括 vfork、spawn、clone 等。

练习

6.3 请描述以下程序在 N 分别为 2 和 4 时的输出。

```
1 int main(void)
2 {
3     int i;
4     for (i = 0; i < N; ++i) {
5         i += fork() ? 1 : 0;
6     }
7     printf("hello\n");
8 }
```

6.3.2 其他进程创建方法

限定场景：vfork

在写时拷贝技术引入之前，fork 的实现是简单地将父进程的内存完整地拷贝一份，因此执行时间较长。为了解决这一问题，BSD 引入了 vfork，作为 fork 在特定场景下的优化。

代码片段 6.17 展示了 `vfork` 的伪代码实现。从接口上来看，`vfork` 与 `fork` 完全一致，但其实现相当于 `fork` 的裁剪版。`vfork` 仍从父进程中创建子进程，但不会为子进程单独创建地址空间，子进程将与父进程共享同一地址空间。这种地址空间共享就带来了潜在的问题：父、子进程中任一进程对内存的修改都会对另一进程产生影响。为了保证正确性，`vfork` 会使父进程在内核中等待，直到子进程调用 `exec` 创建自己独立的地址空间或者退出为止（对应代码中的 `exec_or_exit` 返回值为 `TRUE`）。

代码片段 6.17 `vfork` 的伪代码实现

```
1 int vfork(void)
2 {
3     // 创建一个新的 PCB，用于管理新进程
4     struct process *new_proc = alloc_process();
5     // 虚拟内存初始化：直接使用父进程的页表
6     new_proc->vm_space->pgtbl = curr_proc->vm_space->pgtbl;
7
8     // 内核栈初始化
9     init_kern_stack(new_proc->stack);
10    // 上下文初始化：将父进程 PCB 中的上下文完整拷贝一份
11    copy_context(new_proc->ctx, curr_proc->ctx);
12
13    // 父进程在内核中等待，直到子进程退出或调用 exec
14    while (!exec_or_exit(new_proc))
15        schedule();
16    // 返回
17 }
```

与 `fork` 相比，`vfork` 省去了一次页表拷贝，因此其性能有明显提升。但是，`vfork` 的使用场景相对受限，只适用于进程创建后立即使用 `exec` 的场景。

合二为一：`posix_spawn`

`posix_spawn` 是 POSIX 提供的另一种创建进程的方式，最初是为不支持地址翻译功能的硬件设备设计的。显然，`fork` 无法为这类硬件提供有效支持，因为 `fork` 无法创建两个物理地址完全一样且能独立运行的进程。`posix_spawn` 与 `process_create` 以及 6.2.2 节介绍的 `spawn` 类似，都是从头开始创建进程。不过，Linux 上的 `posix_spawn` 实现与它们存在较大不同。

代码片段 6.18 展示了 `posix_spawn` 的一种实现方式，其可以分为三个步骤。首先，`posix_spawn` 调用 `vfork` 创建一个子进程。之后，其创建的子进程进入一个专门的准备阶段（代码片段 6.18 中的 `prepare_exec`），根据调用者提供的参数对子进程进行配置。最后，子进程调用 `exec`，加载可执行文件并执行。由于子进程调用了 `exec`，父进程也能从 `vfork` 中返回。不难看出，`posix_spawn` 在创建进程后会立即调用 `exec`，因此非常适合使用 `vfork`。在较新版本的 Linux 中，`posix_spawn` 的性能要明显优于 `fork` 和 `exec` 的组合^[4]，该性能提升主要就来自 `vfork` 减少的页表拷贝。

代码片段 6.18 posix_spawn 的伪代码实现

```

1 int posix_spawn(pid_t *pid, const char *path,
2               ...,
3               const posix_spawnattr_t *attrp,
4               char *const argv[],
5               char *const envp[])
6 {
7     // 先执行 vfork 创建一个新进程
8     int ret = vfork();
9     if (ret == 0) {
10         // 子进程：在 exec 之前，根据参数对其进行配置
11         prepare_exec(attrp, ...);
12         // 执行 exec
13         exec(path, argv, envp);
14     } else {
15         // 父进程：将子进程的 pid 设置到传入的参数中
16         *pid = ret;
17         return 0;
18     }
19 }

```

那么 `posix_spawn` 能否完全替代 `fork` 呢？答案是否定的。虽然 `posix_spawn` 提供了 `exec` 之前的准备阶段来配置子进程，但它提供的参数表达能力是有限的，而“`fork+exec`”则有任意多种配置的可能，前面提到的限制子进程资源就无法通过 `posix_spawn` 实现。因此，可以认为 `posix_spawn` 是一种比 `fork` 效率更高但灵活度较低的进程创建方式。

精密控制：`rfork/clone`

由于 `fork` 接口过于简单，因此当应用希望选择性地共享父进程和子进程的部分资源时，`fork` 就爱莫能助了。因此在 20 世纪 80 年代，贝尔实验室的操作系统 Plan 9 首次提出了 `rfork` 接口，支持父进程和子进程之间的细粒度资源共享。而之后的 Linux 操作系统也借鉴了 `rfork`，提出了类似的接口 `clone`。可以认为 `clone` 是 `fork` 的“精密控制”版：同样是通过拷贝的方式创建新进程，`clone` 允许应用通过参数对创建过程进行更多的控制，在功能方面也有一些扩展。

`clone` 的过程与 `fork` 比较相似，也是从已有进程中创建一份拷贝。但相比 `fork` 对父进程的所有结构一概进行复制，`clone` 允许应用传入参数 `flags`，指定应用不需要复制的部分。代码片段 6.19 中举了两个例子：应用可以设定 `CLONE_VM` 以避免复制内存，允许子进程与父进程使用相同的地址空间；同时可以设定 `CLONE_VFORK`，使父进程在内核中等待，直到子进程退出或调用 `exec`。也就是说，在设定了 `CLONE_VFORK` 和 `CLONE_VM` 之后，`clone` 的行为与 `vfork` 相似，如果都不设定则和 `fork` 相似。除以上两个标志位以外，`flags` 还包含其他标志以“精密控制”进程的创建过程，从而使 `clone` 具备了各种功能，其应用场景因此比 `fork` 更加广泛（如用于创建线程）。

代码片段 6.19 clone 的伪代码实现

```

1 int clone(..., int flags, ...)
2 {
3     // 创建一个新的 PCB, 用于管理新进程
4     struct process *new_proc = alloc_process();
5
6     // 如果设置了 CLONE_VM, 则直接使用父进程的页表, 否则拷贝一份
7     if (flags & CLONE_VM) {
8         new_proc->vmSPACE->pgtbl =
9             curr_proc->vmSPACE->pgtbl;
10    } else {
11        new_proc->vmSPACE->pgtbl = alloc_page();
12        copy_vmSPACE(new_proc->vmSPACE,
13                     curr_proc->vmSPACE);
14    }
15
16    // 内核栈初始化
17    init_kern_stack(new_proc->stack);
18    // 上下文初始化: 将父进程 PCB 中的上下文完整拷贝一份
19    copy_context(new_proc->ctx, curr_proc->ctx);
20
21    // 如果设置了 CLONE_VFORK, 则使父进程在内核中等待
22    if (flags & CLONE_VFORK) {
23        while (!exec_or_exit(new_proc))
24            schedule();
25    }
26    // 返回
27 }

```

小知识: Windows 的进程创建——CreateProcess

Windows 采用的进程创建接口为 CreateProcess 系列。其中语义比较简单的是 CreateProcessA, 该函数原型如下:

```

1 #include <ProcessThreadapi.h>
2
3 BOOL CreateProcessA(
4     LPCSTR          lpApplicationName,
5     LPSTR           lpCommandLine,
6     LPSECURITY_ATTRIBUTES lpProcessAttributes,
7     ...             // 其他七个配置参数
8 );

```

CreateProcess 与 posix_spawn 以及前面介绍的 process_create 存在一定的相似之处, 也采取了从头创建进程的方式, 载入参数 lpApplicationName 指定的可执行文件, 并根据其他参数的配置返回用户态执行。但为了支持在创建进程

时对其进行多种多样的配置，CreateProcess 的接口要比 posix_spawn 更加复杂，就算是较为简单的 CreateProcessA 接口也需要传入 10 个参数。

练习

- 6.4 posix_spawn 和 fork 都具有创建进程的功能，但语义有所不同。请分析在以下三种情况下 fork 和 posix_spawn 哪个更适合，并解释原因。
- Web 服务器接收到请求，并创建一个新进程处理该请求。
 - shell 接收到用户输入的 ls 命令，并创建一个新进程来执行该命令。
 - 父进程创建一个子进程，并希望设置共享内存来进行进程之间的通信。

6.4 进程切换

本节主要知识点

- 进程的处理器上下文中包含哪些内容？
- 进程切换包含哪些步骤？
- ChCore 中的进程切换是如何实现的？

现代操作系统通过分时复用的方法，允许不同进程轮番使用处理器，营造出了“多个进程同时执行”的假象。为此，操作系统提出了进程切换机制，从一个进程（原进程）的“上下文”切换到另一个进程（目标进程）的“上下文”执行。在切换过程中，由于进程的重要状态保存在处理器的寄存器中，因此在切换进程前需要保存这些状态（即处理器上下文），以便之后恢复执行。又因为处理器上下文等进程相关的信息都保存在内核中，所以进程切换需要在内核中完成。总的来说，进程切换的主要工作流包括：原进程进入内核态→保存原进程的处理器上下文→切换进程上下文（切换虚拟地址空间和内核栈）→恢复目标进程的处理器上下文→目标进程返回用户态（如图 6.6 所示）。

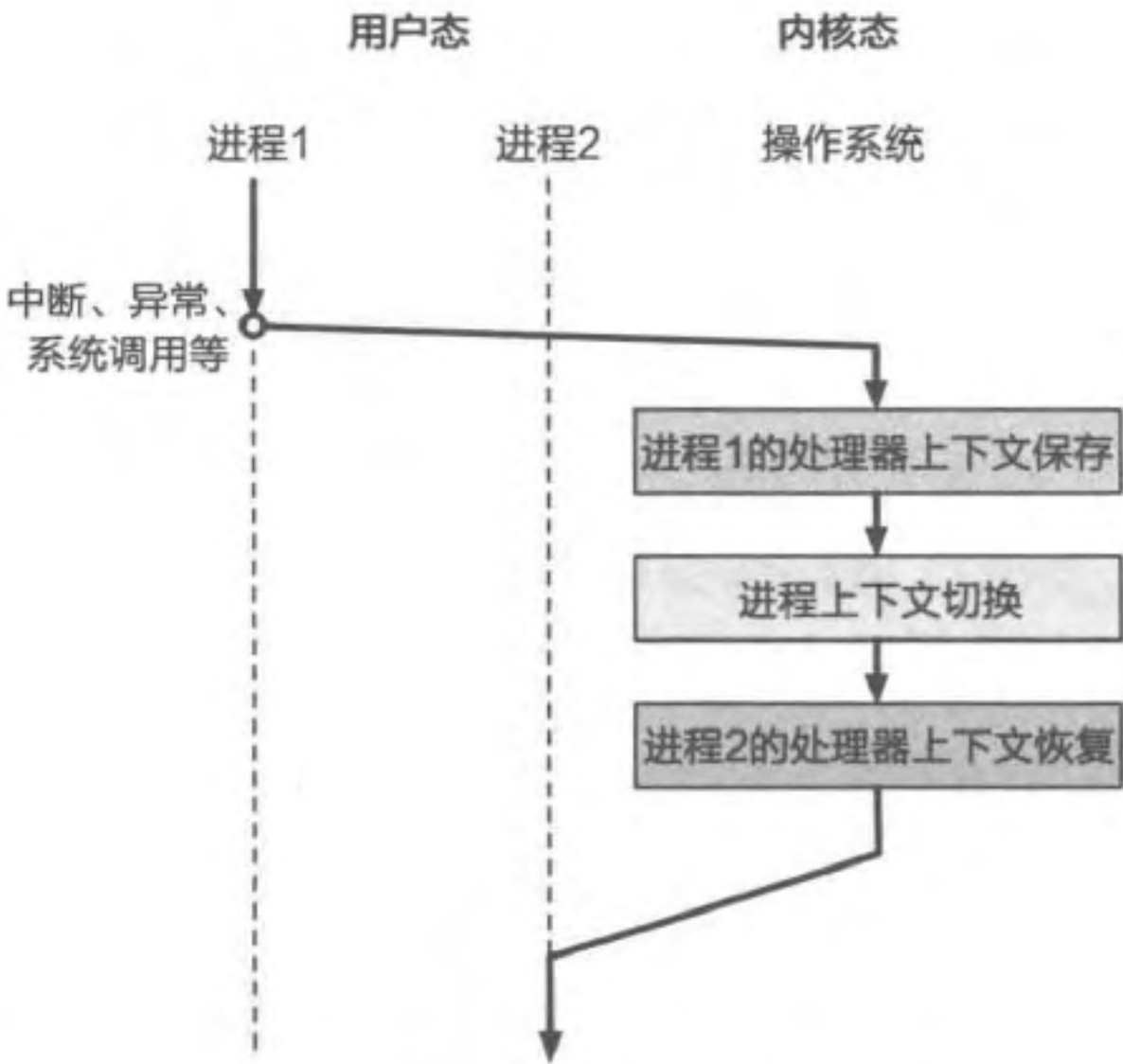


图 6.6 进程切换包含的主要工作流